

GraphQL Blog — Architecture Analysis

Project: GraphQL Blog

Technology: Python, FastAPI, Strawberry GraphQL, SQLAlchemy, PostgreSQL

Location: /home/dominic/Documents/graphql-blog

Report #: 2/20

Architecture Type

GraphQL API with PostgreSQL, Docker Compose orchestration

Layers

GraphQL Schema (Strawberry) → Resolvers → SQLAlchemy ORM → PostgreSQL

Design Patterns

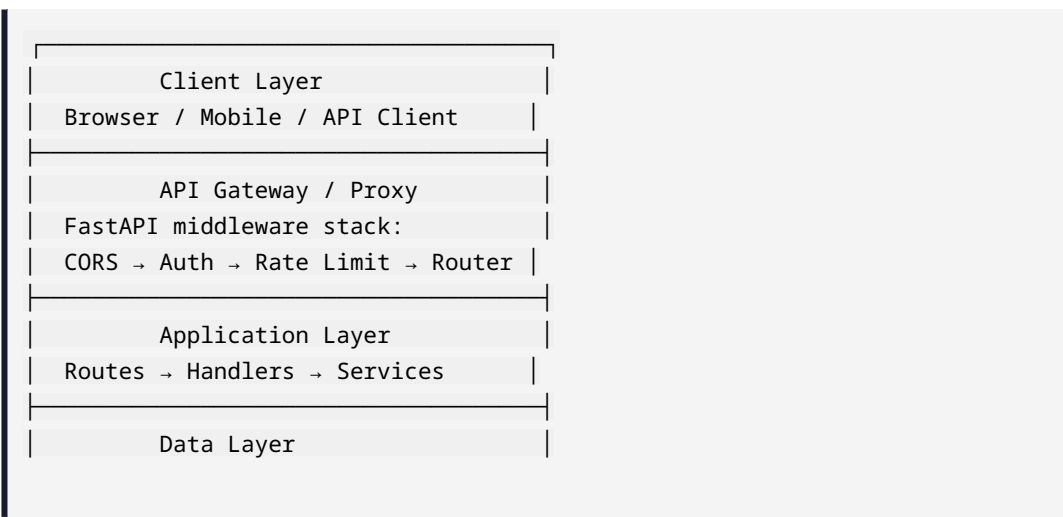
Resolver pattern via Strawberry GraphQL

Scalability

PostgreSQL-backed, horizontally scalable via replicas

System Architecture

Layer Architecture



Component Interaction

GraphQL Blog Component Architecture

- └─ API Layer (FastAPI)
- └─ Service Layer (Business Logic)
- └─ Data Layer (SQLAlchemy + SQLite)
- └─ Client Layer (None)

Request Lifecycle

1. **HTTP Request** arrives at FastAPI
2. **CORS middleware** checks origin headers
3. **Auth dependency** extracts and validates JWT
4. **Rate limiter** checks request count per IP
5. **Router** matches path to handler
6. **Handler** executes business logic
7. **Database session** commits or rolls back
8. **Response** returned to client

Dependency Graph

GraphQL Blog Dependency Flow:

Client → FastAPI → Router → Handler → SQLite → Response

Design Patterns Used

Pattern	Location	Purpose
Dependency Injection	FastAPI Depends()	DB sessions, auth
Repository Pattern	SQLAlchemy models	Data abstraction
Middleware Chain	FastAPI middleware	Cross-cutting concerns
Router Module	APIRouter include	Separation of concerns